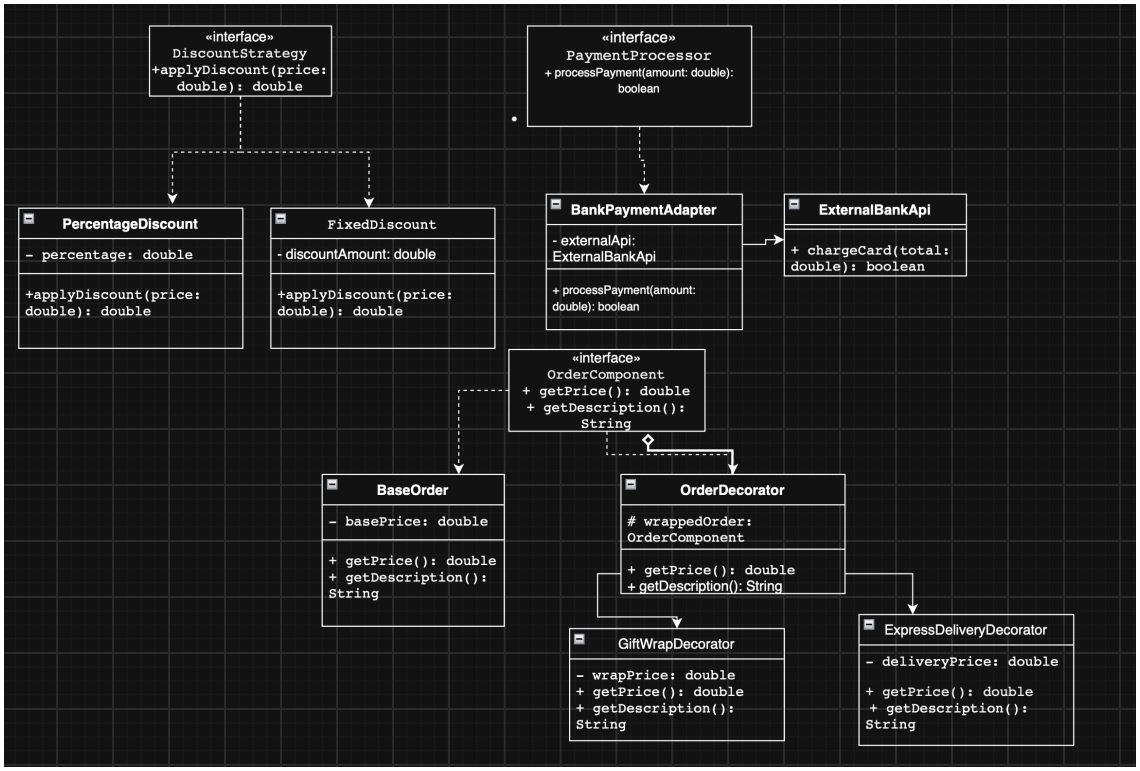
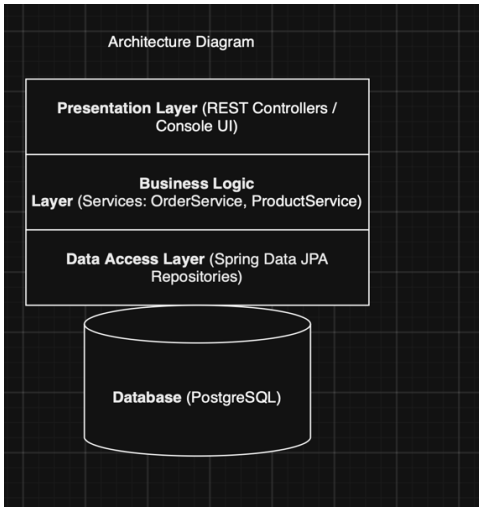
B. Software Design

1.UML



2. Architecture Description The application follows a layered architecture to ensure separation of concerns and maintainability, built on top of the Spring Boot framework:

- **Presentation Layer:** Handled by Spring REST Controllers (and a Console UI client). It processes incoming HTTP requests, validates user input, and returns appropriate responses.
- **Business Logic Layer:** Implemented using Spring Service classes (`@Service`). This layer contains the core application rules, orchestrates data processing, and applies design patterns (Strategy, Decorator, Adapter).
- **Data Access Layer:** Managed by Spring Data JPA interfaces (`@Repository`). It abstracts database interactions,allowing the business logic to perform CRUD operations without writing raw SQL.
- **Domain Model:** Consists of JPA Entities (`@Entity`) representing the core business objects (User, Product, Order,Cart).



3. Explanation of Design Patterns Used To meet the dynamic requirements of the E-Commerce system, three structural and behavioral design patterns were implemented:

- **Strategy Pattern (Discount Calculation):**
  - *Usage:* Applied during the checkout process to dynamically calculate the final order price based on different promotional campaigns.
  - *Implementation:* A common `DiscountStrategy` interface is defined. Concrete classes like `PercentageDiscountStrategy`(e.g., 10% off), `FixedAmountDiscountStrategy` (e.g., $10 off), and `NoDiscountStrategy` implement this interface. The application can switch these strategies at runtime without modifying the core checkout logic.
- **Adapter Pattern (Payment Processing):**
  - *Usage:* Integrates an incompatible third-party payment gateway into the system.
  - *Implementation:* The system expects a `PaymentProcessor` interface. A simulated external API (`ExternalBankApi`) has a completely different structure. The `BankPaymentAdapter` implements `PaymentProcessor` and wraps the `ExternalBankApi`, translating our system's calls into the format expected by the external service.
- **Decorator Pattern (Order Customization):**
  - *Usage:* Dynamically adds optional services (like premium packaging or express delivery) to an order without altering the base `Order` class.
  - *Implementation:* A base `OrderComponent` interface defines the core pricing and description methods. Decorators such as `GiftWrapDecorator` and `ExpressDeliveryDecorator` wrap the base order. When `.getPrice()` or `.getDescription()` is called, the decorators add their specific costs and details to the original order's values.

4. Important Design Decisions

- **Separation of Concerns:** The code strictly separates data retrieval (Repositories) from business rules (Services).Controllers only handle HTTP routing and delegate all heavy lifting to Services.
- **Dependency Injection:** Spring's IoC container is used extensively to inject dependencies (e.g., injecting the correct `DiscountStrategy` into the `OrderService`), making the code highly testable and loosely coupled.